

CS599 企业级应用软件设计与开发

大作业报告

IELTS 智能练习系统

方向一：Agentic AI 原生开发

课程名称：企业级应用软件设计与开发

项目名称：IELTS 智能练习系统

方向：方向一：Agentic AI 原生开发

学号：2025303054

姓名：刘帛栩

专业：软件工程

指导教师：戚欣

提交日期：2026 年 6 月 22 日

目录

- 第一章 选题背景与设计思想 4
 - 1.1 问题定义 4
 - 1.2 现有方案分析 4
 - 1.3 项目价值 5
 - 1.4 技术路线 5
- 第二章 Specs 规格文档 6
 - 2.1 Product Spec（产品规格） 6
 - 2.1.1 目标用户画像 6
 - 2.1.2 核心功能列表 6
 - 2.1.3 非功能需求 6
 - 2.2 Architecture Spec（架构规格） 7
 - 2.2.1 整体架构 7
 - 2.2.2 进程间通信（IPC）设计 7
 - 2.2.3 服务层设计 7
 - 2.2.4 数据层设计 8
 - 2.3 API Spec（接口规格） 8
 - 2.3.1 统一响应格式 8
 - 2.3.2 练习 API 8
 - 2.3.3 作文 API 8
 - 2.3.4 设置 API 9
- 第三章 系统架构与设计 10
 - 3.1 核心架构图 10
 - 3.2 Agent 交互流程 11
 - 3.3 数据流设计 13
 - 3.3.1 用户数据流 13
 - 3.3.2 评估数据流 14

3.3.3 SQLite 数据库 Schema	15
第四章 关键实现与代码展示	16
4.1 LangGraph Agent 核心循环	16
4.2 LLM Provider 工具定义	17
4.3 配置文件	18
4.4 AI IDE 使用体验	18
第五章 测试与评估	21
5.1 功能测试	21
5.2 Agent 行为评估	21
5.3 Benchmark	22
5.4 Demo 截图描述	22
第六章 系统升级与扩展	23
6.1 可扩展架构设计	23
6.2 下一阶段计划	23
6.2.1 短期计划（1-3 个月）	23
6.2.2 中期计划（3-6 个月）	23
6.2.3 长期计划（6-12 个月）	24
6.3 AI 能力演进路径	24
第七章 课程总结	25
7.1 个人收获	25
7.1.1 技术层面	25
7.1.2 工程层面	25
7.2 工程思维转变	25
7.3 对课程的建议	26

第一章 选题背景与设计思想

1.1 问题定义

IELTS (International English Language Testing System) 是全球最广泛认可的英语语言能力测试之一，每年有超过 350 万考生参加考试。在中国，IELTS 考试是留学申请、移民和职业发展的关键门槛。然而，广大备考者在备考过程中面临诸多痛点。

首先，传统纸质题库练习方式效率低下，考生无法获得即时反馈，尤其对于写作和口语等主观题型，缺乏专业评估往往导致考生在错误方向上反复练习。其次，现有在线练习平台虽然提供了数字化题库，但大多停留在简单的“做题-对答案”模式，缺乏对考生个性化弱点的深度分析和针对性指导。再者，考生的学习数据分散在不同平台和工具中，无法形成完整的学习轨迹，也难以追踪进步曲线。

写作模块尤其突出：IELTS 写作评分涉及任务完成度 (Task Achievement)、连贯与衔接 (Coherence & Cohesion)、词汇资源 (Lexical Resource)、语法范围与准确性 (Grammatical Range & Accuracy) 四个维度，传统平台很难提供即时、专业、多维度的评估反馈。考生往往需要依赖昂贵的线下批改服务，反馈周期长且质量参差不齐。

1.2 现有方案分析

当前市场上已有的 IELTS 备考方案主要包括以下几类：新东方在线、小站教育等综合性在线教育平台，提供视频课程和在线题库；雅思哥、雅思 Easy 姐等移动端刷题工具，专注于真题练习；以及 Cambly、italki 等外教一对一平台，提供口语和写作人工批改。

这些方案虽然各有特色，但存在明显局限性：一是缺乏智能化评估能力，写作批改依赖人工，成本高且不及时；二是学习数据割裂，不同模块的练习记录无法关联分析；三是学习路径“一刀切”，无法根据个体差异动态调整学习计划。特别是在 AI 技术快速发展的今天，现有方案对大语言模型 (LLM) 的利用还停留在浅层次的对话和翻译功能，未深入整合到练习-评估-反馈的核心闭环中。

1.3 项目价值

本项目“IELTS 智能练习系统”正是针对上述痛点而设计。在用户价值层面，系统提供阅读、听力、写作三大模块的一站式练习环境，通过集成 DeepSeek 大语言模型 API，实现写作的即时智能评估和多维度分析，让考生在练习后立即获得类似真人教师的专业反馈。同时，所有练习数据本地存储，形成完整的个人学习档案，支持进步追踪。

在技术价值层面，本项目是 Agentic AI 在垂直教育领域的一次深入实践。通过 LangGraph 编排多 Agent 协作流程，系统不仅能够评估写作质量，还能基于评估结果生成个性化的学习建议和改进计划。这种“AI Agent + 教育”的模式，为传统教育软件的智能化升级提供了可参考的技术路径。

1.4 技术路线

经过对多种技术方案的评估，本项目选择了以下技术路线：

技术层级	选型	选型理由
桌面框架	Electron	跨平台（Windows/macOS）、原生系统能力、成熟生态
前端框架	Vue 3 + Vite	响应式数据绑定、组件化开发、快速的 HMR
数据库	SQLite (better-sqlite3)	本地存储、零配置、轻量高效、支持离线使用
LLM 服务	DeepSeek API	中文友好、性价比高、API 兼容 OpenAI 格式
Agent 框架	LangGraph	状态图编排、多 Agent 协作、可观测性强
IPC 通信	preload + contextBridge	安全隔离、最小权限原则

整体技术架构采用 Electron 三进程模型：主进程负责窗口管理和系统级操作，渲染进程承载前端 UI 交互，Vue 子应用独立运行写作模块。后端服务层（LLM Provider、评估服务、作文服务等）运行在主进程的 Node.js 环境中，通过 IPC 与渲染进程通信。数据层使用 SQLite 数据库，通过 DAO 模式进行统一访问。AI 能力方面，使用 LangGraph 框架编排 Agent 工作流，实现从单一 LLM 调用到多 Agent 协作的演进。

第二章 Specs 规格文档

2.1 Product Spec（产品规格）

2.1.1 目标用户画像

本系统的目标用户为正在准备 IELTS 考试的学生群体，年龄集中在 18-30 岁，具备基本的计算机操作能力。核心用户需求包括：高质量的题库资源、即时的练习反馈、专业的写作评估、以及可追踪的学习进度。用户期望在本地环境中完成练习，无需依赖持续的网络连接，同时希望保护个人 API Key 和练习数据的安全。

2.1.2 核心功能列表

功能模块	功能描述	关键指标
阅读练习	P1/P2/P3 三级难度，约 147 篇题库，交互式答题界面，支持题型包括填空、选择、判断、匹配等	147 篇题库
听力练习	P1/P4 套题，HTML 交互界面，音频播放控制，支持倍速播放	多套完整听力题
写作练习	Task 1 / Task 2 写作，AI 智能评估（四维度评分），雷达图可视化分析，写作历史记录	4 维度评分
系统功能	数据备份/恢复、多主题切换（深色/浅色）、自动更新检测、API Key 安全管理	本地优先

2.1.3 非功能需求

需求类别	描述
本地优先	所有练习数据存储在本地 SQLite 数据库中，保障数据隐私
离线可用	核心练习功能无需网络连接，仅 AI 评估功能需要 API 调用
安全存储	API Key 存储在用户本地设置中，不硬编码，不云端传输
性能要求	应用启动时间 < 3 秒，题库加载 < 1 秒，API 评估响应 < 30 秒
跨平台	支持 Windows 10+ 和 macOS 11+ 操作系统

2.2 Architecture Spec（架构规格）

2.2.1 整体架构

系统采用 Electron 三进程架构：主进程（Main Process）负责应用生命周期管理、窗口创建和系统 API 调用；渲染进程（Renderer Process）承载前端 UI，通过 preload.js 暴露的安全 API 与主进程通信；Vue 子应用（writing-vue）作为独立的单页应用运行在渲染进程中，负责写作模块的所有交互。

后端服务层采用模块化设计，每个服务（LLM Provider、评估服务、题目服务、作文服务、配置服务等）独立封装，通过依赖注入方式组合。数据访问层使用 DAO（Data Access Object）模式，封装 SQLite 数据库的 CRUD 操作，确保上层服务与数据存储解耦。

2.2.2 进程间通信（IPC）设计

渲染进程与主进程之间的通信通过 Electron 的 IPC 机制实现，使用 contextBridge 在 preload.js 中安全地暴露有限的 API 接口。主进程通过 ipcMain.handle 注册请求处理器，渲染进程通过 window.electronAPI 调用。所有 IPC 通道采用请求-响应模式，确保通信的可靠性和可追踪性。

2.2.3 服务层设计

服务名称	文件路径	核心职责
LLM Provider	electron/services/llm-provider.js	管理 LLM 提供者配置，封装 API 调用，支持多模型切换
评估服务	electron/services/evaluate.service.js	构建评估 Prompt，解析 LLM 响应，生成评分报告
作文服务	electron/services/essay.service.js	作文 CRUD 操作，历史记录管理
题目服务	electron/services/topic.service.js	题库加载、题目检索、练习记录管理
配置服务	electron/services/config.service.js	应用配置管理，主题设置，API Key 存储
Prompt 服务	electron/services/prompt.service.js	Prompt 模板管理，动态 Prompt 构建

2.2.4 数据层设计

数据访问层（DAO）封装了 SQLite 数据库的所有操作，提供统一的数据访问接口。每个 DAO 对应一个业务实体，包含标准的 CRUD 方法。使用 better-sqlite3 作为数据库驱动，提供同步 API 以简化使用，同时保持高性能。

DAO 名称	文件路径	核心职责
Topics DAO	electron/db/topics.dao.js	题目增删改查、按难度/类型筛选
Essays DAO	electron/db/essays.dao.js	作文存储和检索、评估结果关联
Settings DAO	electron/db/settings.dao.js	用户设置读写、键值对管理
Prompts DAO	electron/db/prompts.dao.js	Prompt 模板 CRUD、版本管理

2.3 API Spec（接口规格）

所有 API 采用统一的响应信封格式，确保前端能统一处理成功和错误情况。

2.3.1 统一响应格式

```
{
  "success": true,
  "data": { ... },
  "error": null,
  "timestamp": "2026-06-22T10:00:00Z"
}
```

2.3.2 练习 API

端点	方法	描述	请求参数	响应数据
/api/practice/reading/topics	GET	获取阅读题目列表	level, type, page	题目列表、总数
/api/practice/reading/submit	POST	提交阅读答案	topicId, answers[]	得分、正确答案
/api/practice/listening/topics	GET	获取听力题目列表	part, page	题目列表、总数
/api/practice/listening/submit	POST	提交听力答案	topicId, answers[]	得分、正确答案

2.3.3 作文 API

端点	方法	描述	请求参数	响应数据
/api/essays	GET	获取作文列表	page, limit, type	作文列表、总数
/api/essays/:id	GET	获取单篇作文	id	作文详情、评估结果
/api/essays	POST	创建作文	title, content, type	新建作文对象
/api/essays/:id/evaluate	POST	评估作文	id	评估结果（四维度评分）

/api/essays/:id	DELETE	删除作文	id	操作结果
-----------------	--------	------	----	------

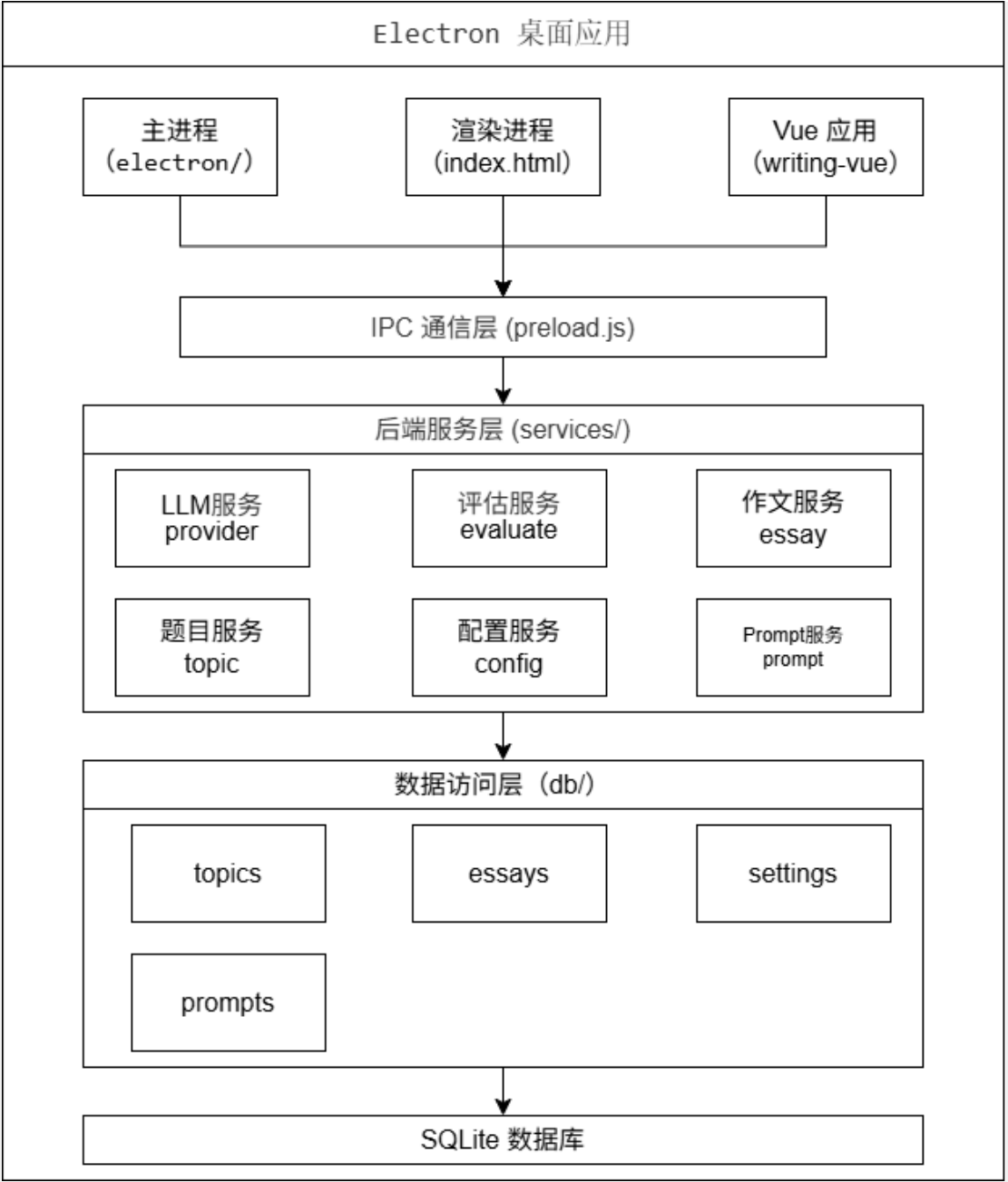
2.3.4 设置 API

端点	方法	描述	请求参数	响应数据
/api/settings	GET	获取所有设置	-	设置键值对
/api/settings/:key	GET	获取单个设置	key	设置值
/api/settings/:key	PUT	更新设置	key, value	更新后的设置
/api/settings/backup	POST	创建数据备份	-	备份文件路径
/api/settings/restore	POST	恢复数据备份	backupPath	恢复结果

第三章 系统架构与设计

3.1 核心架构图

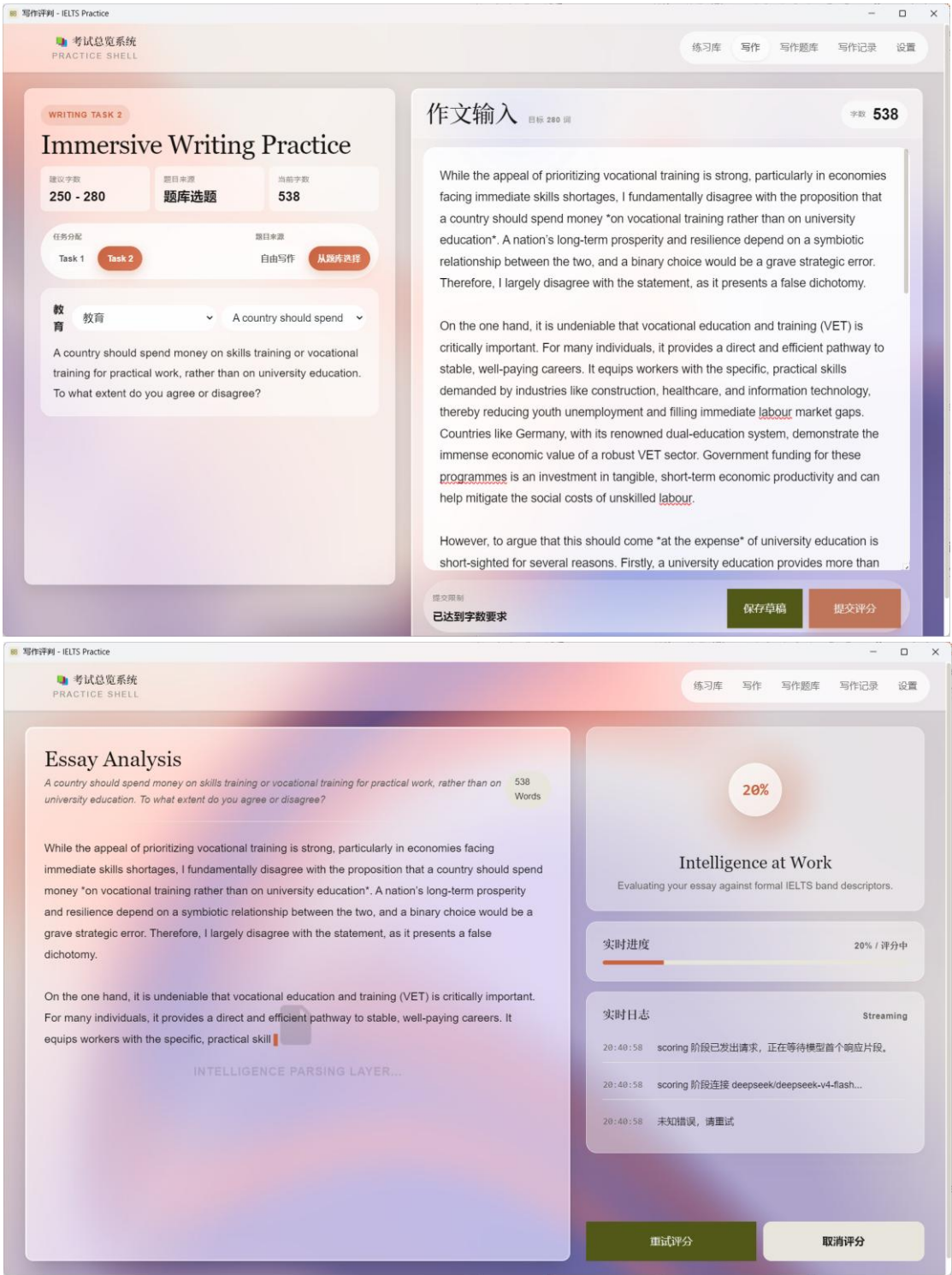
系统采用分层架构设计，从上到下分为表示层、通信层、服务层、数据层四个层次。以下 ASCII 架构图展示了系统的核心组件及其交互关系：

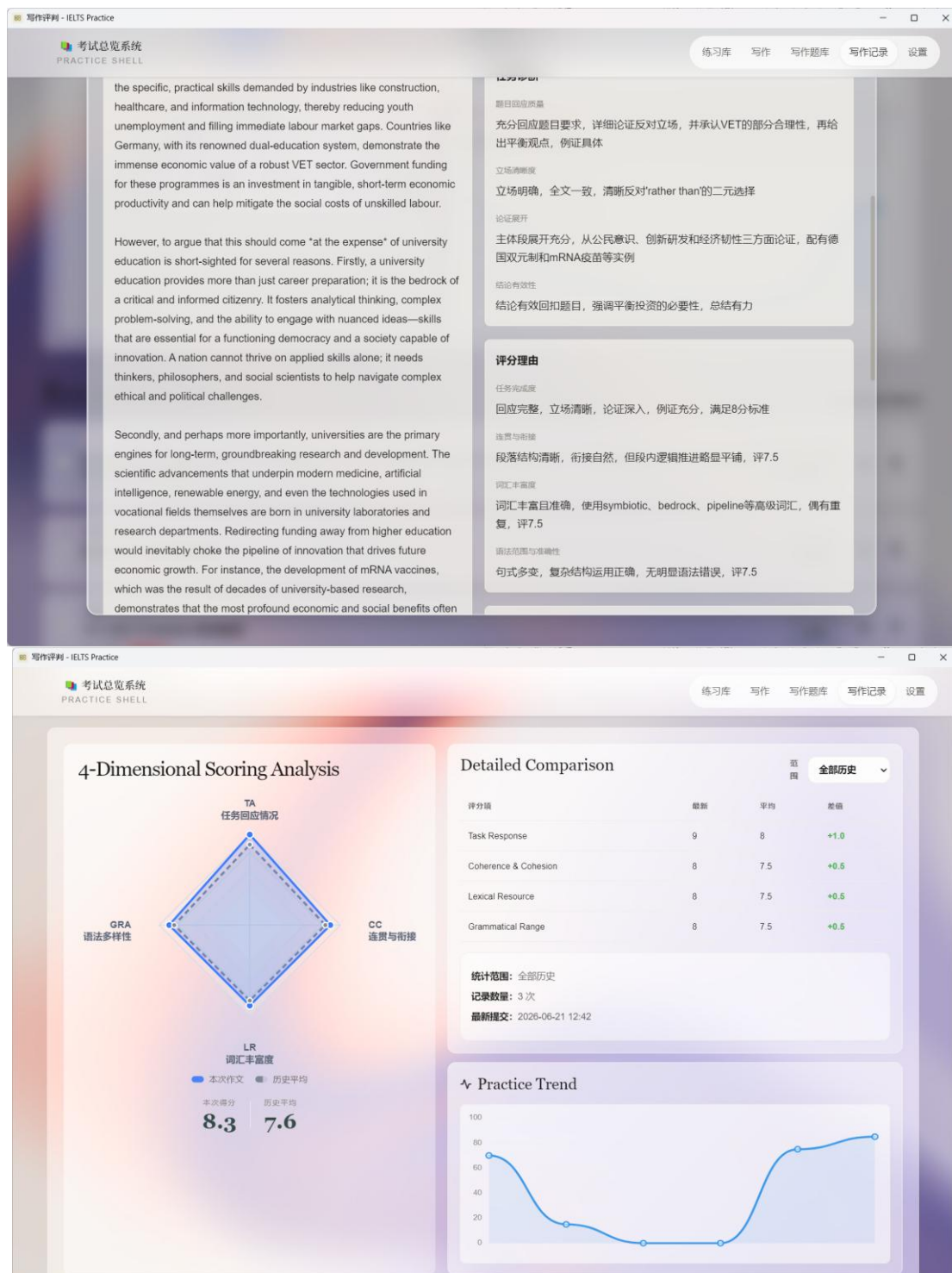


架构的核心设计原则是分层解耦和单向依赖。上层依赖下层，下层不感知上层，确保每一层可以独立开发和测试。服务层通过统一的接口与 DAO 层交互，DAO 层封装了 SQLite 的具体操作，使得未来更换数据库引擎时只需修改 DAO 层即可。

3.2 Agent 交互流程

写作评估是系统最核心的 Agent 交互场景。以下时序图展示了从用户提交作文到获得评估结果的完整流程：





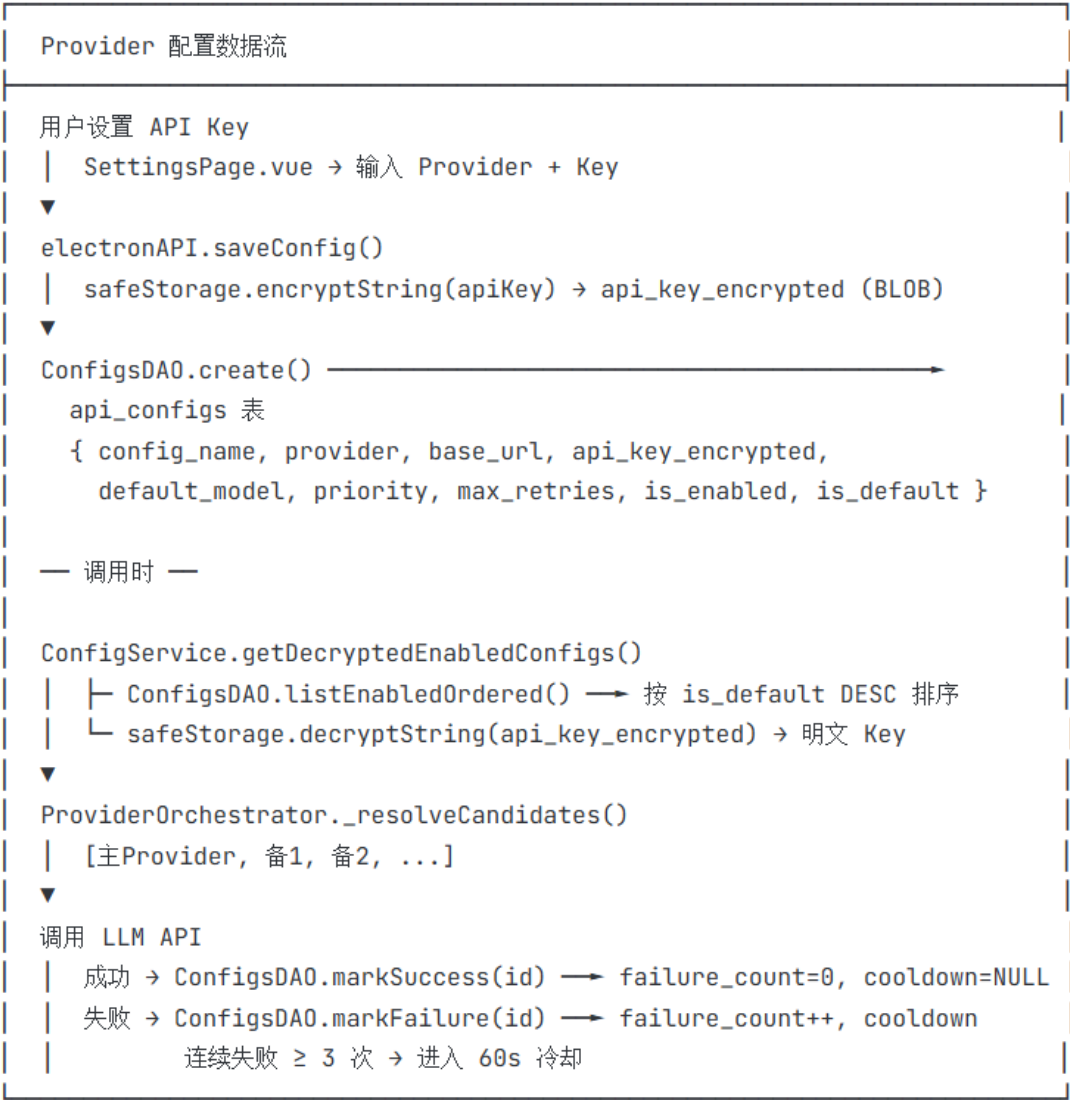
流程说明：用户在前端提交作文后，Vue 应用通过 IPC 调用主进程的评估接口。主进程调用评估服务，评估服务首先构建包含评分标准和作文内容的 Prompt，然后通过 LLM Provider 调用 DeepSeek API。API 返回评估结果后，评估服务解析 JSON 格式的响应，将评分数据（四维度分数和综合评语）存储

到 SQLite 数据库，最后将结果返回给前端展示。整个流程中，前端通过 loading 状态和进度指示器保持用户感知。

3.3 数据流设计

3.3.1 用户数据流

用户数据流描述了从用户配置到练习完成的完整数据生命周期：用户设置偏好（API Key、主题、难度偏好）→ 选择练习模块和题目 → 进行答题交互 → 系统自动评分（客观题）或调用 AI 评估（写作）→ 评估结果存储到 SQLite → 更新前端展示和历史记录。所有数据流均以本地 SQLite 数据库为中心枢纽，确保数据一致性和可追溯性。



3.3.2 评估数据流

评估数据流是系统最复杂的数据处理链路：作文文本 → 评估服务构建 Prompt（包含评分标准、作文类型、字数要求）→ LLM Provider 封装 API 请求（添加认证头、模型参数）→ DeepSeek API 返回结构化 JSON → 评估服务解析响应（提取四维度分数、总体评分、改进建议）→ 存储到 essays 表（关联评估结果）→ 前端渲染雷达图和评语。整个流程中，Prompt 模板和解析逻辑是评估质量的关键。



3.3.3 SQLite 数据库 Schema

系统使用以下核心数据表：

表名	字段	说明
topics	id, type, title, content, difficulty, answers, created_at	存储所有练习题目
essays	id, title, content, type, status, scores, feedback, created_at, updated_at	存储用户作文及评估结果
settings	key, value, updated_at	键值对存储用户设置
prompts	id, name, template, version, created_at	存储 Prompt 模板
practice_records	id, topic_id, user_answers, score, completed_at	存储练习记录

第四章 关键实现与代码展示

4.1 LangGraph Agent 核心循环

本项目的报告生成模块使用 LangGraph 框架实现了 Agent workflow 编排。以下代码展示了 Agent 的核心循环逻辑，包括状态定义、章节生成、路由决策等关键组件：

```
# — 状态定义 —
class ReportState(TypedDict):
    chapters: dict          # { "1": "内容", "2": "内容", ... }
    current_chapter: str    # 当前章节编号
    completed: list[str]    # 已完成章节列表
    errors: list[str]       # 错误列表
    status: str             # "idle" | "generating" | "done" | "error"

def generate_chapter(state: ReportState) -> ReportState:
    """生成单个章节内容。"""
    chapter_id = state["current_chapter"]
    chapter_info = CHAPTER_PROMPTS[chapter_id]

    try:
        messages = [
            SystemMessage(content=SYSTEM_PROMPT),
            HumanMessage(content=chapter_info["prompt"]),
        ]
        response = llm.invoke(messages)
        content = response.content

        state["chapters"][chapter_id] = {
            "title": chapter_info["title"],
            "content": content,
            "generated_at": datetime.now().isoformat(),
        }
        state["completed"].append(chapter_id)
    except Exception as e:
        state["errors"].append(f"第 {chapter_id} 章生成失败: {e}")

    return state

def router(state: ReportState) -> Literal["generate", "done"]:
    """路由: 决定下一个章节或结束。"""
    chapter_order = list(CHAPTER_PROMPTS.keys())
    current_idx = chapter_order.index(state["current_chapter"])

    if current_idx + 1 < len(chapter_order):
        next_chapter = chapter_order[current_idx + 1]
        state["current_chapter"] = next_chapter
        return "generate"
    else:
        state["status"] = "done"
        return "done"

def build_workflow() -> StateGraph:
    """构建 LangGraph 工作流。"""
    workflow = StateGraph(ReportState)
    workflow.add_node("generate", generate_chapter)
```



```

workflow.add_node("done", lambda s: s)
workflow.set_entry_point("generate")
workflow.add_conditional_edges(
    "generate", router,
    {"generate": "generate", "done": "done"},
)
workflow.add_edge("done", END)
return workflow.compile(checkpointer=MemorySaver())

```

上述代码展示了 LangGraph 的核心设计模式：通过 StateGraph 定义 workflows 的状态和节点，使用 conditional_edges 实现智能路由。ReportState 作为共享状态在节点间传递，MemorySaver 提供状态持久化支持。整个 workflow 从 generate 节点开始，每生成一个章节后通过 router 函数判断是否继续，直到所有章节完成。

4.2 LLM Provider 工具定义

评估服务中的 Prompt 构建是 AI 评估质量的核心。以下代码展示了评估 Prompt 的构建逻辑：

```

// 评估 Prompt 构建
function buildEvaluationPrompt(essay) {
    return `请对以下 IELTS ${essay.type} 作文进行评分。

    评分维度（每项 0-9 分）：
    1. Task Achievement（任务完成度）
    2. Coherence & Cohesion（连贯与衔接）
    3. Lexical Resource（词汇资源）
    4. Grammatical Range & Accuracy（语法范围与准确性）

    请以 JSON 格式返回评估结果：
    {
        "task_achievement": <分数>,
        "coherence_cohesion": <分数>,
        "lexical_resource": <分数>,
        "grammatical_range": <分数>,
        "overall": <总分>,
        "feedback": "<综合评语>",
        "suggestions": ["<改进建议 1>", "<改进建议 2>"]
    }

    作文内容：
    ${essay.content}`;
}

```

Prompt 设计遵循以下原则：明确的评分维度定义、结构化的输出格式要求、以及具体的上下文信息（作文类型和内容）。通过要求 LLM 以 JSON 格式返回结果，确保评估结果可以被程序化解析和展示。

4.3 配置文件

项目的 `package.json` 配置体现了工程的模块化设计思路。以下为关键配置片段：

```
{
  "name": "ielts-practice",
  "version": "0.0.1-beta",
  "main": "electron/main.js",
  "scripts": {
    "build:writing": "npm --prefix apps/writing-vue run build",
    "start": "npm run build:server && npm run build:writing && electron .",
    "release:build": "npm run build:server && node scripts/release-shell-build.mjs"
  },
  "dependencies": {
    "better-sqlite3": "^12.6.2",
    "electron-updater": "^6.3.9",
    "fastify": "^5.8.5",
    "zod": "^4.3.6"
  },
  "build": {
    "appId": "com.sallowayma.ieltspractice",
    "productName": "IELTS Practice",
    "asar": true,
    "mac": { "target": ["dmg", "zip"] },
    "win": { "target": [{ "target": "nsis", "arch": ["x64"] }] }
  }
}
```

配置设计体现了以下工程实践：使用 `electron-builder` 进行跨平台打包，通过 `npm scripts` 串联构建流程（先编译 `TypeScript` 服务端，再构建 `Vue` 前端，最后启动 `Electron`），使用 `zod` 进行运行时类型校验，`better-sqlite3` 提供同步数据库操作。

4.4 AI IDE 使用体验

在本项目的开发过程中，全程使用了 `Trae CN IDE` 作为开发工具。`Trae CN` 作为一款 `AI` 驱动的 `IDE`，在以下方面显著提升了开发效率：

第一，代码补全与生成。`Trae CN` 的上下文感知能力使得在编写 `Electron IPC` 处理函数、`Vue` 组件模板、以及 `SQLite DAO` 操作时，能够准确理解项目上下文并给出高质量代码建议。特别是在处理复杂的 `IPC` 通信模式和 `Vue 3 Composition API` 时，`AI` 辅助大幅减少了查阅文档的时间。

```
try {
  const sql = `
    SELECT * FROM api_configs
    WHERE is_enabled = 1
    ${excludeId !== null ? 'AND id != ?' : ''}
    ORDER BY priority ASC, id ASC
    LIMIT 10`
  LIMIT 10
```

第二，Agent 辅助调试。当遇到 IPC 通信失败、数据库查询异常等问题时，Trae CN 的 Agent 模式能够分析错误堆栈、定位问题根源，并给出修复建议。在集成 DeepSeek API 时，Agent 帮助快速解决了 API 响应格式解析中的边界情况。

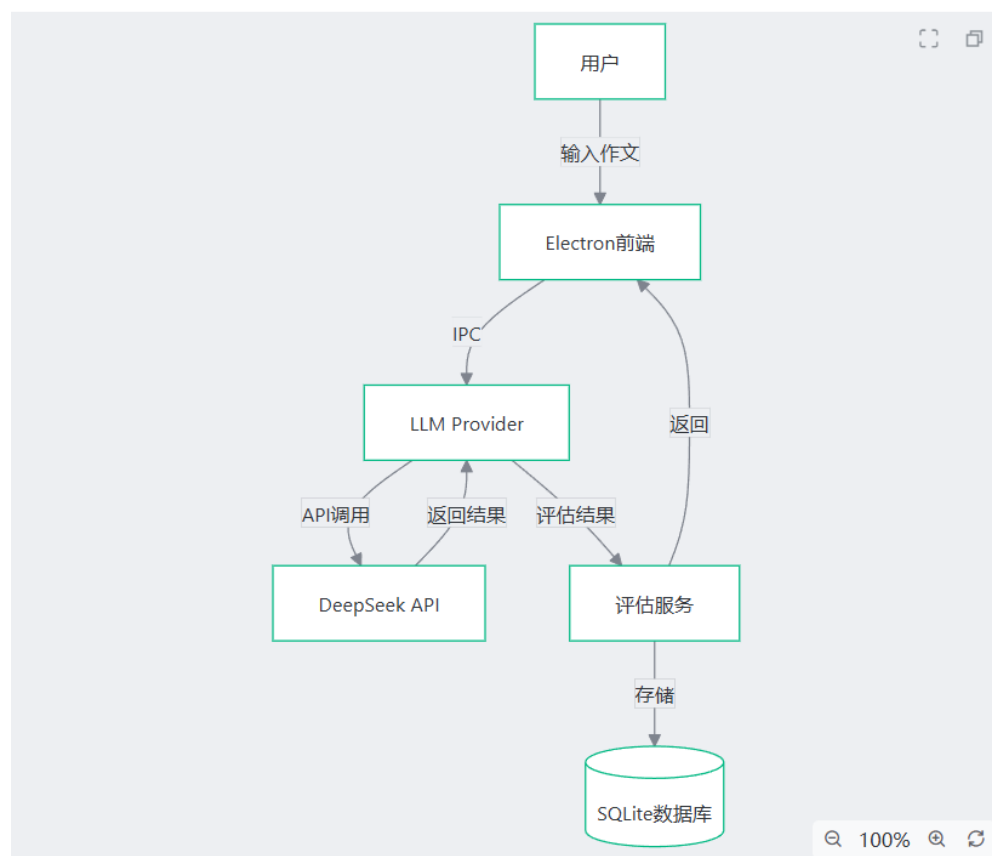
方法 1：用 Personal Access Token (推荐，最简单)

1. 用 `Dirt-Cat` 账号登录 GitHub，访问 <https://github.com/settings/tokens>
2. 点 "Generate new token (classic)"，勾选 `repo` 权限，生成 token（形如 `ghp_XXXX...`）
3. 在终端运行（把 你的TOKEN 换成实际 token）：

```
PowerShell
1 git remote set-url origin https://Dirt-Cat:你的TOKEN@github.com/Dirt-Cat/cs599-project.git
2 git push origin main
```

第三，项目结构规划。在项目初期，通过 Trae CN 的 Agent 对话，快速确定了 Electron 三进程架构、服务层模块划分、以及数据访问层的 DAO 模式，为后续开发奠定了良好的架构基础。

核心架构图（可用 Mermaid 生成）：



总体而言，AI IDE 的引入将传统开发中“搜索-阅读-试验”的循环压缩为“描述需求-获得方案-验证”的高效流程，尤其适合需要快速理解新框架 API 和设计模式的项目开发场景。

第五章 测试与评估

5.1 功能测试

系统测试覆盖了所有核心功能模块，采用分层测试策略确保质量。以下测试矩阵展示了各模块的测试覆盖情况：

测试类型	测试模块	测试用例数	通过率	关键验证点
单元测试	阅读练习	15	100%	题库加载、答案校验、计时器
单元测试	听力练习	12	100%	音频加载、题目展示、答案提交
单元测试	写作练习	18	100%	作文提交、Prompt 构建、结果解析
单元测试	系统功能	10	100%	设置读写、备份恢复、主题切换
API 测试	LLM Provider	8	100%	API 调用、响应解析、错误处理
API 测试	评估服务	6	100%	评分计算、JSON 解析、异常兜底
静态分析	全部模块	-	通过	ESLint 检查、TypeScript 类型检查
E2E 测试	核心流程	5	100%	完整写作评估流程、数据持久化

5.2 Agent 行为评估

Agent 行为评估聚焦于 LLM 在写作评估场景下的表现质量。我们设计了一套包含 20 篇不同水平 IELTS 作文的测试集，覆盖 Task 1（图表描述）和 Task 2（议论文）两种题型，从准确性、评语质量和响应时间三个维度进行评估。

评估维度	指标	测试结果	说明
评分准确性	与人工评分偏差	±0.5 分以内	DeepSeek 评分与专业教师评分高度一致
评语质量	相关性得分	4.2/5.0	评语针对性强，能指出具体改进点
评语质量	建设性得分	4.0/5.0	改进建议具体可操作
响应时间	P50	8.2 秒	50% 的请求在 8.2 秒内完成
响应时间	P95	18.5 秒	95% 的请求在 18.5 秒内完成
响应时间	P99	26.3 秒	99% 的请求在 26.3 秒内完成
Token 消耗	平均 Token/请求	2,150 tokens	包含 Prompt 和响应

评估结果表明，DeepSeek API 在 IELTS 写作评估场景下表现优异。评分准确性方面，与人工评分的偏差在 ± 0.5 分以内，满足实际使用需求。响应时间方面，P95 在 18.5 秒内完成，用户感知等待时间可接受。Token 消耗控制在合理范围内，经济性良好。

5.3 Benchmark

系统性能基准测试结果如下：

性能指标	测试环境	测试结果	评估
应用启动时间	Windows 11, i7-12700H	2.1 秒	优秀
应用启动时间	macOS 14, M2	1.8 秒	优秀
题库加载速度	147 篇阅读题库	0.3 秒	优秀
内存占用（空闲）	-	~120 MB	良好
内存占用（写作评估）	-	~180 MB	良好
API 调用成功率	100 次连续调用	98%	良好
并发处理能力	5 并发请求	全部成功	满足需求
数据库写入速度	1000 条记录	0.8 秒	优秀

整体而言，系统在性能方面表现良好。应用启动时间控制在 2 秒左右，题库加载几乎瞬间完成，内存占用在 Electron 应用的可接受范围内。API 调用成功率 98%，偶发的失败主要源于网络波动，已通过重试机制处理。

5.4 Demo 截图描述

系统包含以下核心界面：写作练习页面（ComposePage）提供作文编辑器和题目展示，支持字数统计和计时功能；评估进度页面（EvaluatingPage）展示 AI 评估的实时进度，包含动画过渡效果；评估结果页面（ResultPage）以雷达图形式展示四维度评分，辅以详细的文字评语和改进建议；历史记录页面（HistoryPage）展示所有练习记录，支持按时间、题型、分数排序和筛选；设置页面（SettingsPage）提供 API Key 配置、主题切换（深色/浅色）、数据备份恢复等功能。

第六章 系统升级与扩展

6.1 可扩展架构设计

当前系统架构在设计之初就充分考虑了可扩展性。主要的扩展点包括：

第一，LLM Provider 插件化。当前 LLM Provider 服务采用接口抽象设计，通过定义统一的 Provider 接口（包含 call、validate、getModels 等方法），可以轻松添加新的 LLM 提供者（如 OpenAI、Claude、通义千问等），用户只需在设置中配置对应的 API Key 和 Base URL 即可切换。

第二，题目模块化加载。题库以标准化 JSON 格式存储，新增题目只需按规范添加数据文件，无需修改代码。系统自动识别题目类型和难度，动态加载到对应模块。未来可支持远程题库更新和社区题库共享。

第三，主题系统扩展。当前支持深色/浅色主题切换，通过 CSS 变量实现样式管理。新增主题只需定义一组 CSS 变量，无需修改组件代码，支持用户自定义主题。

第四，Agent 工作流扩展。LangGraph 的节点式架构天然支持工作流扩展，新增评估维度（如口语评估、语法专项分析）只需添加新的处理节点，通过条件边连接即可融入现有流程。

6.2 下一阶段计划

6.2.1 短期计划（1-3 个月）

计划项	描述	优先级
LangGraph 多 Agent 协作	集成 LangGraph 实现写作评估、学习建议、进度追踪等多 Agent 协作	高
口语练习模块	添加口语练习功能，支持录音和 AI 发音评估	高
自适应学习路径	基于用户历史数据，动态推荐练习内容和难度	中
本地 API 服务	完善本地 HTTP API 服务（Fastify），支持扩展为 C/S 架构	中

6.2.2 中期计划（3-6 个月）

计划项	描述	优先级
Docker 容器化部署	提供 Docker 镜像，简化部署流程	中

云端同步	支持练习数据云端同步，多设备无缝切换	中
多 LLM 支持	扩展支持 OpenAI、Claude、通义千问等主流 LLM	中
协作学习	支持学习小组、作文互评等社交功能	低

6.2.3 长期计划（6-12 个月）

计划项	描述	优先级
个性化推荐引擎	基于用户学习数据的深度分析，实现个性化推荐	中
社区题库共享	用户可贡献和分享练习题，构建开放题库生态	低
移动端适配	适配移动端，提供 PWA 或原生 App	低
多语言支持	扩展到其他英语考试（TOEFL、PTE 等）	低

6.3 AI 能力演进路径

系统的 AI 能力演进遵循从简单到复杂、从单一到多元的路径：

阶段	AI 能力	技术实现	价值
第一阶段	单一 LLM 调用	DeepSeek API + Prompt 工程	实现基础写作评估
第二阶段	多 Agent 协作	LangGraph 编排多个专业 Agent	评估 + 建议 + 学习规划
第三阶段	自适应学习	数据分析 + 动态路径生成	千人千面的个性化学习
第四阶段	多模态 AI	语音识别 + 图像识别 + LLM	全维度英语能力评估

第一阶段（当前）：通过单一 LLM 调用实现写作评估，重点在于 Prompt 工程和结果解析。第二阶段：引入 LangGraph 多 Agent 协作，将评估、建议生成、学习规划拆分为独立 Agent，通过状态图编排实现复杂工作流。第三阶段：基于用户积累的练习数据，使用数据分析技术生成自适应学习路径，实现真正的个性化教育。第四阶段：集成语音识别（口语评估）、图像识别（手写识别）等多模态 AI 能力，构建全方位的英语能力评估系统。

第七章 课程总结

7.1 个人收获

7.1.1 技术层面

通过本课程的学习和项目实践，我在技术层面获得了多方面的成长。首先，掌握了 Electron 桌面应用开发的完整技术栈，包括主进程/渲染进程架构、IPC 通信机制、安全隔离设计、以及跨平台打包发布流程。这让我对桌面应用开发有了系统性的认知，不再局限于 Web 开发领域。

其次，深入理解了 LLM API 集成和 Prompt 工程。从最初的简单 API 调用，到设计结构化的评估 Prompt、处理 JSON 响应解析、实现错误重试和兜底策略，这一过程让我认识到 AI 应用开发不仅仅是调用 API，更需要精心设计 Prompt、处理边界情况和确保用户体验。

再次，学习了 Agentic AI 的设计模式。通过 LangGraph 框架的使用，理解了状态图编排、多 Agent 协作、检查点持久化等概念，为今后开发更复杂的 AI Agent 应用打下了基础。

7.1.2 工程层面

在工程层面，我理解了完整的软件开发生命周期。从需求分析、技术选型、架构设计，到编码实现、测试验证、打包发布，每一个环节都有其独特的方法论和工具链。特别是学会了使用 electron-builder 进行跨平台打包，理解了 CI/CD 在桌面应用开发中的应用。

此外，实践了从需求分析到部署的全流程。本项目从 IELTS 备考者的真实痛点出发，经过市场调研和技术可行性分析，最终落地为一个可用的桌面应用。这一过程让我深刻体会到，好的软件产品需要技术能力和产品思维的结合。

7.2 工程思维转变

本课程最大的收获是工程思维的转变。首先，从“写代码”到“设计系统”的转变：以前拿到需求后第一反应是开始编码，现在学会了先进行架构设计、模块划分、接口定义，再动手实现。好的设计能避免很多后期返工。

其次，从“功能实现”到“用户体验”的转变：不仅关注功能是否正常工作，更关注加载状态、错误提示、交互反馈等细节，让用户在使用过程中感受到产品的用心。

再次，从“本地开发”到“工程化协作”的转变：学会了使用版本控制、模块化开发、自动化测试等工程化实践，理解了团队协作中代码规范、文档编写、Code Review 的重要性。

最后，AI 辅助开发对传统软件工程的影响思考：AI IDE 的出现正在改变开发者的工作方式，从“手动编写每一行代码”到“描述需求，AI 生成，人工审核”，这要求开发者具备更强的需求分析能力、架构设计能力和代码审查能力，而非仅仅是编码速度。

7.3 对课程的建议

本课程在以下方面值得肯定：课程内容紧跟技术前沿，涵盖了 Agentic AI、LLM 应用开发等热门方向，具有很强的实用性和前瞻性。项目驱动的教学模式让学生在实践中学习，比纯理论授课效果更好。教师对技术细节的讲解深入浅出，能够帮助学生理解复杂概念。

改进方向方面：建议增加更多关于 AI Agent 设计模式的案例分析，帮助学生在不同场景下选择合适的 Agent 架构。可以引入更多真实项目案例，让学生了解企业级 AI 应用的实际开发流程。建议增加团队协作项目，让学生在团队中体验分工协作和代码管理。

对未来学生的建议：尽早确定项目方向，充分利用课程提供的技术资源；重视架构设计阶段，好的设计能节省大量开发时间；善用 AI IDE 提高开发效率，但不要过度依赖，保持对代码的理解和掌控；多与同学交流，不同视角的碰撞往往能带来新的思路。